

**INF-07b** Vylepším algoritmus podle zvoleného hlediska a zobecním řešení pro širší třídu problémů.

Na test máš 30 minut. Pracuj samostatně, odpovědi a postup zapisuj čitelně na papír nebo do dokumentu a hotové řešení nahraj do portfolia na Google Disku (fotka, sken nebo soubor). Povolené: počítač s Pythonem (Colab, IDLE) – programy můžeš spouštět a upravovat. Nepoužívej AI asistenta. Do portfolia přilož i upravené programy (soubor nebo odkaz na Colab).

1. Funkce vrací seznam pozic, na kterých je v seznamu největší prvek. Co program vypíše? Kolik porovnání zhruba provede pro seznam délky  $n$ , když víš, že volání `max(s)` projde celý seznam? Vylepši funkci tak, aby byla rychlejší, a urči počet kroků vylepšené verze.
2. Obě verze funkce z úlohy 1 jsi změřil (časy v sekundách): původní –  $n = 1\,000$ : 0,007; 2 000: 0,028; 4 000: 0,113; vylepšená – 0,00003; 0,00006; 0,00011. Doložují měření zlepšení? Jak se chová čas každé verze při zdvojnásobení  $n$ ? Co musíš ještě ověřit, aby bylo vylepšení opravdu hotové?
3. Funkce spočítá, kolikrát lekce dostala v hodnocení 5 hvězdiček. Zobecni ji na funkci `pocet_vyskytu(s, hodnota)`, která spočítá výskyty libovolné hodnoty v libovolném seznamu. Co vrátí volání `pocet_vyskytu([5, 3, 5, 4, 5], 4)`, `pocet_vyskytu(["ano", "ne", "ano", "nevím"], "ano")` a `pocet_vyskytu([], 5)`?

```
def pocet_petek(hodnoceni):  
    pocet = 0  
    for h in hodnoceni:  
        if h == 5:  
            pocet += 1  
    return pocet  
  
print(pocet_petek([5, 3, 5, 4, 5]))
```

4. Funkce vrací mince hladovým postupem (vždy co největší mince). Původně měla sadu mincí 20, 10, 5, 2, 1 pevně zapsanou, teď je zobecněná a sada mincí je parametr. Co program vypíše? Je výsledek druhého volání nejlepší možný? Co z toho plyne pro zobecňování algoritmů?

```
def vrat(castka, mince):  
    vysledek = []  
    for m in mince:                # mince od největší  
        while castka >= m:  
            castka -= m  
            vysledek.append(m)  
    return vysledek  
  
print(vrat(38, [20, 10, 5, 2, 1]))  
print(vrat(6, [4, 3, 1]))
```

5. Program vygeneruje milion náhodných hodnot a vypíše největší hodnotu a průměr. Přepiš ho tak, aby nepotřeboval seznam všech hodnot. Proč je nová verze úspornější a jak ověříš, že dává stejný výsledek?

```
import random  
random.seed(5)  
n = 1000000  
hodnoty = []  
for i in range(n):  
    hodnoty.append(random.randint(0, 100))  
print(max(hodnoty), sum(hodnoty) / n)
```

6. Program počítá cenu jízdenky podle počtu zón. Co vypíše pro 3 zóny a co pro 5 zón? Přepiš ho přehledněji tak, aby šlo přidat další zónu změnou jediného řádku: ceny ulož do seznamu a výpočet dej do funkce s parametrem.

```
zony = int(input("Počet zón: "))
if zony == 1:
    cena = 24
elif zony == 2:
    cena = 32
elif zony == 3:
    cena = 40
elif zony == 4:
    cena = 48
print("Cena:", cena, "Kč")
```

7. Uveď tři různá hlediska, podle kterých lze algoritmus vylepšit. Ke dvěma z nich uveď příklad, kdy zlepšení podle jednoho hlediska zhorší jiné.

Test slouží k doložení výstupu do portfolia. Nehodnotí se body ani známkou – rozhoduje, jestli splníš kritéria na konci dokumentu.

## Klíč a řešení

- [1, 3]. Funkce volá max(s) v každém průchodu cyklem – asi  $n \cdot n = n^2$  porovnání (pro  $n = 10\,000$  asi  $10^8$ ). Vylepšení: maximum spočítat jednou před cyklem ( $m = \max(s)$ ) a v cyklu porovnávat s  $m$  – asi  $2n$  kroků (pro  $n = 10\,000$  asi  $2 \cdot 10^4$ ). – ověřuje: nalezení zbytečné práce v algoritmu a vylepšení jeho rychlosti**
  - Cyklus proběhne  $n \times$ , v každém průchodu max(s) projde  $n$  prvků: asi  $n^2$  kroků.
  - Úprava: řádek  $m = \max(s)$  před cyklem, podmínka `if s[i] == m`.
  - Nová verze:  $n$  kroků na maximum +  $n$  porovnání v cyklu =  $2n$ .
  - Výsledek se nezmění – pro `[4, 9, 2, 9, 5]` obě verze vrátí `[1, 3]`.
- Ano: původní verze je při zdvojnásobení  $n$  asi  $4 \times$  pomalejší (kvadratická), vylepšená asi  $2 \times$  (lineární); pro 4 000 prvků je vylepšená asi 1 000 $\times$  rychlejší. Ověřit: že obě verze dávají stejné výsledky na stejných testech (i krajní případy – jeden prvek, víc maxim, záporná čísla) a měřit víckrát a pro více velikostí vstupu. – ověřuje: doložení vylepšení měřením a testy**
- def pocet\_vyskytu(s, hodnota): pocet = 0; for x in s: if x == hodnota: pocet += 1; return pocet. Volání vrátí 1; 2; 0. Pevnou hodnotu 5 nahradil parametr, funkce funguje pro čísla i texty a i pro prázdný seznam. – ověřuje: zobecnění řešení – parametr místo pevné hodnoty**
- [20, 10, 5, 2, 1] a [4, 1, 1]. Druhý výsledek není nejlepší:  $6 = 3 + 3$  stačí dvě mince. Hladový postup je správný pro české mince, ale ne pro každou sadu – zobecněné řešení je třeba otestovat na širší třídě vstupů a uvést, pro jaké vstupy platí (případně použít algoritmus, který zkouší víc možností). – ověřuje: meze zobecnění algoritmu na širší třídu problémů**
- V cyklu rovnou přičítat k součtu a porovnávat s dosavadním maximem: nej = None; soucet = 0; v cyklu `h = random.randint(0, 100)`, `soucet += h`, `if nej is None or h > nej: nej = h`; nakonec `print(nej, soucet / n)`. Místo milionu uložených čísel (desítky MB) stačí dvě proměnné. Stejný výsledek ověřím spuštěním obou verzí se stejným `random.seed(5)` – obě vypíší 100 49.996983. – ověřuje: vylepšení algoritmu z hlediska paměti a ověření shody výsledků**
  - Seznam hodnoty zbytečně drží všech  $10^6$  čísel, přitom maximum i součet jde počítat průběžně.
  - Průběžně: `soucet += h` a porovnání `h` s dosavadním maximem `nej`.
  - Paměť: 2 proměnné místo  $10^6$  prvků seznamu; čas zůstává úměrný  $n$ .
  - Kontrola: stejné semínko `random.seed(5)` → stejné náhodné hodnoty → obě verze vypíší 100 49.996983.
- Pro 3 zóny „Cena: 40 Kč“, pro 5 zón skončí chybou NameError (proměnná cena nevznikla). Vylepšení: `CENY = [None, 24, 32, 40, 48]`; funkce `cena_jizdenky(zony, ceny=CENY)` vrátí `ceny[zony]`, když `1 <= zony < len(ceny)`, jinak None (neplatný vstup). Nová zóna = jedno číslo navíc v seznamu; funkci lze použít i s jiným ceníkem (jiné město). – ověřuje: vylepšení čitelnosti a zobecnění programu (data místo řady podmínek)**
- Hlediska: rychlost (počet kroků, čas), paměť, čitelnost a udržitelnost (případně obecnost). Příklady: předpočítaná tabulka výsledků zrychlí výpočet, ale zabere paměť; ukládání jen součtu místo seznamu šetří paměť, ale pak nejde spočítat medián; „chytře“ zrychlený kód bývá hůř čitelný; zobecnění na libovolná data přidá kontroly a program může zpomalit. – ověřuje: volba hlediska vylepšení a jeho důsledky**

## Kritéria splnění výstupu

- Najde v algoritmu zbytečnou práci a vylepší ho podle zvoleného hlediska – rychlost, paměť, čitelnost (aspoň 2 ze 3 úloh 1, 5 a 6).
- Zlepšení doloží počtem kroků nebo měřením pro několik velikostí vstupu a ověří, že se výsledky nezměnily (úlohy 1, 2 a 5).
- Zobecní řešení nahrazením pevné hodnoty parametrem a funkci použije pro jiná data (úlohy 3 a 6).
- Posoudí meze zobecnění – pro jakou třídu vstupů řešení platí (úloha 4) – a ví, že vylepšení podle jednoho hlediska může zhoršit jiné (úloha 7).

## Pro učitele: varianty a přizpůsobení

Obměny: bublinkové řazení bez ukončení při žádné výměně (vylepšení break), součet  $1 + 2 + \dots + n$  cyklem  $\times$  vzorcem, test prvočíselnosti se všemi děliteli  $\times$  děliteli do  $\sqrt{n}$ ; v úloze 3 zobecnění lineárního vyhledávání na funkci, která vrátí všechny pozice. Pro silnější žáky: ať zlepšení v úloze 1 sami naměří v Colabu pro tři velikosti vstupu a výsledky zapíší do tabulky. Pro slabší: v úloze 1 napovědět, že max(s) je sám cyklus. Podkladem do portfolia je odkaz na Colab s původní a vylepšenou verzí, testy a měřeními.

Výstup se probírá v lekcích: 1.+2. H60 Efektivita algoritmů (týden 30)